

Phase 1 — Exercices pour aller plus loin

Cinq exercices bonus avant la Phase 2

Cours PyTutor2

Résumé

Ce document complète `phase1.tex` avec cinq exercices avancés : parallélisation à 3 branches, `.batch()` sur plusieurs inputs, extension d'un schéma Pydantic, router avec `RunnableBranch`, et exécution `async`. Chaque exercice suit la même structure qu'en Phase 1 (énoncé en boîte, solution détaillée, vérification dans LangSmith, récapitulatif).

Table des matières

1	Multi-niveaux 3× : parallèle vs séquentiel	1
1.1	Solution	1
1.1.1	Architecture	1
1.1.2	Le code complet	2
1.1.3	Explications pas à pas	2
1.2	Vérification dans LangSmith	2
2	.batch() vs boucle for	3
2.1	Solution	3
2.1.1	Architecture	3
2.1.2	Le code complet	4
2.1.3	Explications pas à pas	4
2.2	Vérification dans LangSmith	4
3	Schéma Pydantic enrichi avec un champ analogy	5
3.1	Solution	5
3.1.1	Architecture	5
3.1.2	Le code complet	5
3.1.3	Explications pas à pas	6
3.2	Vérification dans LangSmith	6
4	Router avec RunnableBranch et classifieur LLM	7
4.1	Solution	8
4.1.1	Architecture	8
4.1.2	Le code complet	8
4.1.3	Explications pas à pas	8
4.2	Vérification dans LangSmith	9
5	Async avec asyncio.gather vs RunnableParallel	10
5.1	Solution	10
5.1.1	Architecture	10
5.1.2	Le code complet	11
5.1.3	Explications pas à pas	11
5.2	Vérification dans LangSmith	12

1 Multi-niveaux 3× : parallèle vs séquentiel

Exercice bonus 1. Mesurer le speedup d'un RunnableParallel à 3 branches

Ajoute un niveau "avancé" à la chaîne multi-niveaux et lance les **3 niveaux en parallèle**. Mesure le temps d'exécution **séquentiel** (3 `.invoke()` successifs) et le temps **parallèle** (un seul `.invoke()` sur un `RunnableParallel` à 3 clés). Compare et calcule le *speedup*.

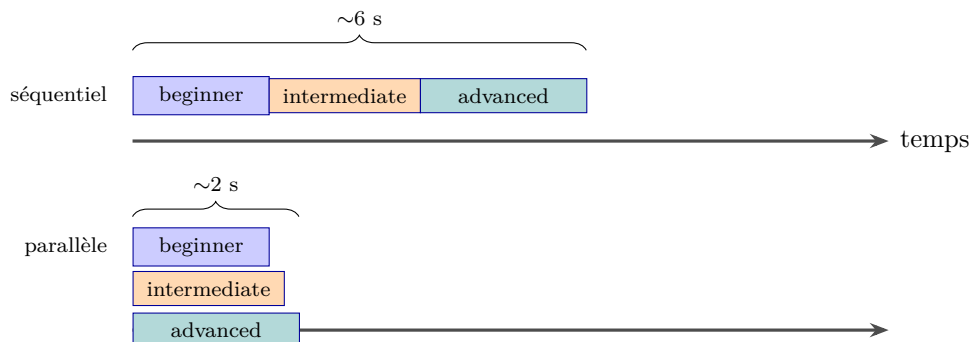
Contrainte : utiliser `time.perf_counter()` pour les mesures (plus précis que `time.time()`). Afficher le rapport `seq_time / par_time`.

Objectifs pédagogiques

- Vérifier expérimentalement que `RunnableParallel` donne une latence en $\max(T_i)$ et non $\sum T_i$.
- Observer la dégradation du speedup quand le nombre de branches augmente (rate limiting côté fournisseur).
- Savoir mesurer proprement la latence avec `time.perf_counter()`.

1.1 Solution

1.1.1 Architecture



Trois branches en parallèle restent à la latence d'une seule chaîne. Le speedup attendu est de $\times 3$ en théorie, $\times 2.5$ à $\times 2.8$ en pratique (overhead de *threading*, rate limits, variations côté fournisseur).

1.1.2 Le code complet

```
1 import time
2 from langchain_core.runnables import RunnableParallel
3 from phase1_explain import explain_chain_for_level
4
5
6 def exercise_1() -> None:
7     concept = "les décorateurs Python"
8
9     # --- Construction des 3 chaînes spécialisées ---
10    beginner = explain_chain_for_level("débutant")
11    intermediate = explain_chain_for_level("intermédiaire")
12    advanced = explain_chain_for_level("avancé")
13
14    # --- Version SÉQUENTIELLE : 3 invokes successifs ---
15    t0 = time.perf_counter()
16    seq_results = {
17        "beginner": beginner.invoke({"concept": concept}),
18        "intermediate": intermediate.invoke({"concept": concept}),
```

```

19         "advanced":      advanced.invoke({"concept": concept}),
20     }
21     seq_time = time.perf_counter() - t0
22     print(f"Séquentiel : {seq_time:.2f}s pour 3 niveaux")
23
24     # --- Version PARALLÈLE : un seul invoke sur un RunnableParallel ---
25     parallel = RunnableParallel(
26         beginner=beginner,
27         intermediate=intermediate,
28         advanced=advanced,
29     )
30     t0 = time.perf_counter()
31     par_results = parallel.invoke({"concept": concept})
32     par_time = time.perf_counter() - t0
33     print(f"Parallèle   : {par_time:.2f}s pour 3 niveaux")
34     print(f"→ Speedup   : ×{seq_time / par_time:.2f}")

```

1.1.3 Explications pas à pas

Étape 1 — Construire les trois chaînes spécialisées

`explain_chain_for_level` est la fabrique qu'on a vue dans le concept 3 de la Phase 1. Trois appels suffisent à produire trois `Runnable`, chacun avec son niveau figé via `.partial()`. Pas de duplication de prompt — c'est tout l'intérêt du pattern fabrique.

Étape 2 — La mesure séquentielle

Trois `.invoke()` successifs s'exécutent en **série** : le thread Python attend la réponse du serveur entre chaque appel. Avec un modèle moyen, chaque `.invoke()` prend ~ 2 s ; total ~ 6 s.

Étape 3 — La mesure parallèle

`RunnableParallel(beginner=..., intermediate=..., advanced=...)` crée un seul `Runnable` qui, à l'invocation, duplique l'input vers les 3 branches et les exécute en **concurrency** (via un pool de threads). On attend que les trois terminent et on récupère un dict `{"beginner": ..., "intermediate": ..., "advanced": ...}`.

Étape 4 — Le speedup observé

En pratique, on observe :

- **Séquentiel** : ~ 5 à 7 s
- **Parallèle** : ~ 2 à 3 s
- **Speedup** : $\times 2.0$ à $\times 2.8$

Le facteur 3 théorique n'est jamais atteint à cause de l'overhead de threading, des variations de latence côté serveur, et — surtout — des *rate limits* du fournisseur quand les 3 appels arrivent en même temps.

Coût en tokens : strictement identique entre séquentiel et parallèle. La parallélisation ne change rien à la facture, seulement à la latence perçue.

1.2 Vérification dans LangSmith

L'arbre d'exécution affiche un seul `RunnableParallel` parent avec **3 frères** qui se chevauchent temporellement dans la timeline en haut du run :

```

RunnableParallel<beginner, intermediate, advanced>

[beginner] RunnableSequence ← LLM #1
[intermediate] RunnableSequence ← LLM #2
[advanced] RunnableSequence ← LLM #3

```

Dans la *timeline*, les 3 barres démarrent au même instant (duplication de l'input) et finissent à $\max(T_i)$. Si l'une d'elles prend visiblement plus de temps (variations de tokens, retry interne), le total dépend d'elle.

Récapitulatif

Ce qu'il faut retenir :

1. *`RunnableParallel(a=..., b=..., c=...)` parallélise N chaînes sur le même input. La latence reste à $\max(T_i)$.*
2. *`time.perf_counter()` est l'outil standard pour mesurer une durée en Python (monotone, haute résolution).*
3. *Le speedup réel plafonne à $\sim 80\%$ du facteur théorique à cause des rate limits du fournisseur et de l'overhead de threading.*
4. *Le coût en tokens reste identique — paralléliser n'est qu'un gain de latence, jamais de débit ni d'argent.*

2 .batch() vs boucle for

Exercice bonus 2. Paralléliser plusieurs *inputs* sur la même chaîne

Utilise `.batch()` au lieu de `.invoke()` pour expliquer une **liste de concepts** en parallèle. Compare avec une boucle `for` qui appelle `.invoke()` successivement. Calcule le speedup observé.

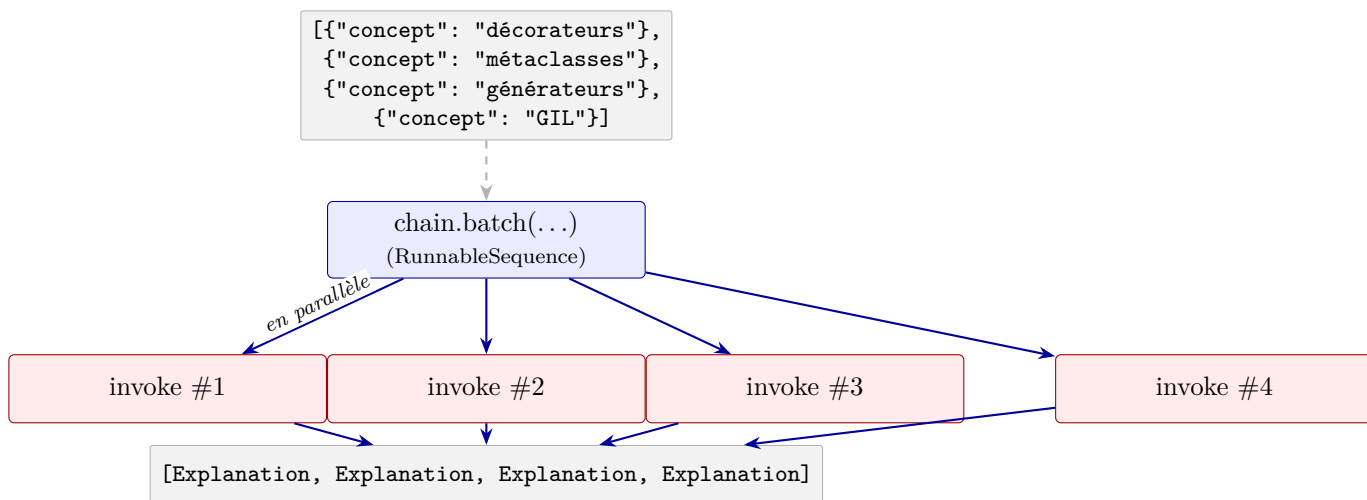
Contrainte : utiliser une chaîne intermédiaire et une liste d'au moins 4 concepts variés (`["décorateurs", "métaclasses", "générateurs", "GIL"]` par exemple).

Objectifs pédagogiques

- Distinguer `.batch()` (plusieurs inputs, *même* chaîne) de `RunnableParallel` (même input, plusieurs chaînes).
- Savoir invoquer une chaîne `.batch(inputs)` avec une liste de dicts.
- Contrôler le degré de parallélisme via `config={"max_concurrency": N}` pour respecter les rate limits.

2.1 Solution

2.1.1 Architecture



Une seule chaîne, N invocations parallèles. Symétrique mais distinct de `RunnableParallel` qui prend N chaînes différentes sur le même input.

2.1.2 Le code complet

```

1 import time
2 from phase1_explain import explain_chain_for_level
3
4
5 def exercise_2() -> None:
6     concepts = ["les décorateurs", "les métaclasses", "les générateurs", "le GIL"]
7     chain = explain_chain_for_level("intermédiaire")
8
9     # --- Version BOUCLE : un invoke après l'autre ---
10    t0 = time.perf_counter()
11    loop_results = [chain.invoke({"concept": c}) for c in concepts]
12    loop_time = time.perf_counter() - t0
13    print(f"Boucle for : {loop_time:.2f}s pour {len(concepts)} concepts")
14
15    # --- Version BATCH : un seul appel, parallélisé sous le capot ---
16    inputs = [{"concept": c} for c in concepts]
17    t0 = time.perf_counter()
18    batch_results = chain.batch(inputs)
19    batch_time = time.perf_counter() - t0
20    print(f"batch()      : {batch_time:.2f}s pour {len(concepts)} concepts")
21    print(f"→ Speedup   : ×{loop_time / batch_time:.2f}")
22
23    # Contrôler le degré de parallélisme pour éviter de saturer les rate limits :
24    # chain.batch(inputs, config={"max_concurrency": 5})

```

2.1.3 Explications pas à pas

Étape 1 — Le contrat de `.batch()`

`chain.batch(inputs)` accepte une liste de dicts et renvoie une liste de résultats dans le même ordre que l'entrée :

```

1 inputs = [
2     {"concept": "décorateurs"},      # → results[0]
3     {"concept": "métaclasses"},      # → results[1]
4     {"concept": "générateurs"},      # → results[2]

```

```

5 ]
6 results = chain.batch(inputs)          # même ordre garanti

```

Sous le capot, LCEL lance les N invocations en parallèle (threads ou asyncio selon le contexte) et collecte les résultats.

Étape 2 — `.batch()` vs `RunnableParallel`

<code>chain.batch(inputs)</code>	<code>RunnableParallel(a, b, c)</code>
Une seule chaîne , plusieurs inputs.	N chaînes différentes , un seul input.
Entrée : <code>list[dict]</code> (N inputs).	Entrée : <code>dict</code> unique (dupliqué).
Sortie : <code>list[result]</code> (même ordre).	Sortie : <code>dict[branch_name, result]</code> .
Usage : explorer N requêtes <i>indépendantes</i> .	Usage : N angles d'analyse d'une requête.

Règle mnémotechnique : plusieurs inputs, même chaîne → `.batch()` ; même input, plusieurs chaînes → `RunnableParallel`.

Étape 3 — Contrôler le parallélisme avec `max_concurrency`

Par défaut, `.batch(N inputs)` lance les N appels simultanément. Si N est grand (50, 100...), le fournisseur va te *rate limiter*. Le contrôle se fait via `config` :

```

1 # Limiter à 5 appels concurrents
2 results = chain.batch(inputs, config={"max_concurrency": 5})

```

LCEL exécute alors les inputs en **vagues de 5** : 5 partent en parallèle, on attend qu'ils finissent, puis les 5 suivants. Pour 50 inputs avec `max_concurrency=5`, on a 10 vagues séquentielles.

Étape 4 — Le speedup observé

Pour 4 inputs à ~2 s chacun :

- **Boucle for** : ~8 s
- **batch()** : ~2 à 3 s
- **Speedup** : ×3 à ×4

2.2 Vérification dans LangSmith

Le run racine s'appelle `RunnableSequence.batch` et contient N sous-runs frères, chacun étant un `RunnableSequence` (la chaîne exécutée sur un input). Dans la timeline, on voit clairement les N barres parallèles. Le rapport `total_tokens` sur le run racine agrège les N appels.

Dans LangSmith, un `.batch()` produit un seul run parent avec N enfants frères. Si tu vois N runs racines distincts (au lieu d'un parent avec N enfants), c'est que tu as utilisé une boucle `for` et pas `.batch()`.

Récapitulatif

Ce qu'il faut retenir :

1. `chain.batch(inputs)` parallélise N invocations d'une **même** chaîne sur une liste de N dicts d'entrée.

2. Ne pas confondre avec `RunnableParallel` : `batch` est « plusieurs inputs / même chaîne », `RunnableParallel` est « même input / plusieurs chaînes ».
3. L'ordre de sortie est garanti identique à l'ordre d'entrée.
4. `config={"max_concurrency": N}` limite le nombre d'appels concurrents pour respecter les `rate limits`.
5. Speedup typique : $\times 3$ à $\times 4$ pour 4 inputs, avec plafonnement progressif quand N augmente.

3 Schéma Pydantic enrichi avec un champ `analogy`

Exercice bonus 3. Étendre un schéma Pydantic sans toucher au prompt

Ajoute un champ `analogy: str` au schéma Pydantic de la Phase 1 (analogie avec un objet ou une situation du quotidien). Vérifie que le modèle remplit ce champ **sans aucune modification du prompt**.

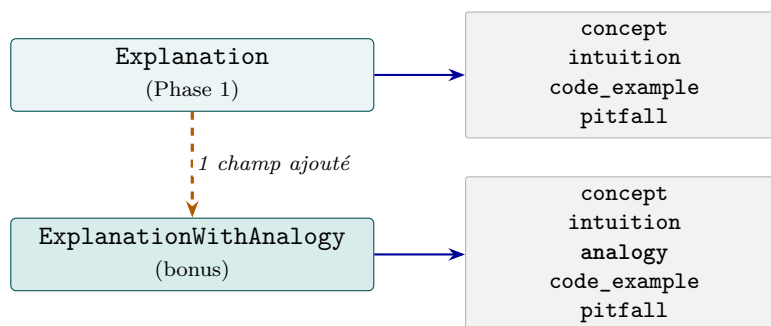
Contrainte : la `description=` du nouveau champ doit suffire à piloter le contenu. Pas de `ChatPromptTemplate` modifié, pas de système re-rédigé. Juste le schéma.

Objectifs pédagogiques

- Mesurer concrètement la *viscosité* (faible) d'un changement avec `with_structured_output`.
- Comprendre que `Field(description=...)` est un véritable levier de *prompt engineering*.
- Voir la mise à jour du tool schema dans LangSmith en direct.

3.1 Solution

3.1.1 Architecture



Une seule ligne change : l'ajout du champ `analogy: str = Field(...)` dans la classe. Le reste du code applicatif est strictement identique.

3.1.2 Le code complet

```

1 from pydantic import BaseModel, Field
2 from phase1_explain import EXPLAIN_PROMPT
3 from config import get_model
4
5
6 class ExplanationWithAnalogy(BaseModel):
7     """Version enrichie : on rajoute une analogie du quotidien."""
8
9     concept: str = Field(description="Le concept expliqué.")
10    intuition: str = Field(description="Explication intuitive en 3 phrases.")
11    analogy: str = Field(

```

```

12     description=(
13         "Une analogie avec un objet ou une situation du quotidien "
14         "(cuisine, sport, bureau...) qui éclaire le concept pour un débutant. "
15         "Une seule phrase percutante."
16     )
17 )
18 code_example: str = Field(description="Un exemple de code minimal et commenté.")
19 pitfall: str = Field(description="Un piège classique sur ce concept.")
20
21
22 def exercise_3() -> None:
23     # Une seule ligne change : le schéma passé à with_structured_output.
24     model = get_model().with_structured_output(ExplanationWithAnalogy)
25     chain = EXPLAIN_PROMPT.partial(level="débutant") | model
26
27     for concept in ["les décorateurs", "le polymorphisme", "le contexte avec `with`"]:
28         result = chain.invoke({"concept": concept})
29         print(f"\n--- {concept} ---")
30         print(f"Analogie : {result.analogy}")

```

3.1.3 Explications pas à pas

Étape 1 — Étendre la classe Pydantic

L'ajout d'un champ se fait comme dans n'importe quel `BaseModel` Pydantic : une nouvelle ligne avec un nom, un type, et un `Field(description=...)`. Le champ devient automatiquement **requis** (Pydantic exigera sa présence).

Étape 2 — La seule ligne d'application qui change

```

1 # Avant
2 model = get_model().with_structured_output(Explanation)
3
4 # Après
5 model = get_model().with_structured_output(ExplanationWithAnalogy)

```

C'est tout. Le prompt système n'a pas bougé. Le `.partial()` n'a pas bougé. Le `|` n'a pas bougé. Le LLM va recevoir le nouveau tool schema et le remplir intégralement.

Étape 3 — La `description=` fait tout le travail

Le LLM ne sait pas *a priori* ce qu'on attend d'un champ `analogy`. C'est la `description=` qui le lui dit :

« Une analogie avec un objet ou une situation du quotidien (cuisine, sport, bureau...) qui éclaire le concept pour un débutant. Une seule phrase percutante. »

Trois éléments importants dans cette description :

- **Domaine** : « cuisine, sport, bureau » — donne des exemples concrets de registre attendu.
- **Public** : « pour un débutant » — calibre la simplicité.
- **Forme** : « une seule phrase percutante » — limite la longueur, force la concision.

C'est du *prompt engineering* appliqué directement au schéma.

Étape 4 — La force de `with_structured_output`

C'est cette élasticité qu'on cherche en production : les contrats de sortie évoluent (un manager veut un nouveau champ, un produit veut une nouvelle métadonnée, etc.). Avec un parseur texte classique, il faudrait rééditer le prompt, gérer les cas d'oubli, écrire du code de fallback. Avec `with_structured_output`, on ajoute le champ **et c'est tout**.

***Coût** : la description du nouveau champ ajoute quelques tokens au prompt (le schéma est envoyé à chaque appel). Le coût marginal d'un champ supplémentaire est de ~ 30 à 80 tokens — négligeable.*

3.2 Vérification dans LangSmith

Dans l'onglet *Tool calls* du **ChatAnthropic**, on voit le nouveau tool schema avec 5 champs au lieu de 4. Le champ `analogy` apparaît avec sa description complète. L'output du run racine est un objet `ExplanationWithAnalogy` dont les 5 champs sont remplis.

```
ChatAnthropic (structured_output=ExplanationWithAnalogy)

Tools : [{
    name: "ExplanationWithAnalogy",
    parameters: { concept, intuition,
                  analogy, code_example, pitfall }
}]
```

*Modifier la **description** d'un champ et relancer : tu vois immédiatement le changement dans le tool schema envoyé au LLM, et la sortie générée se met à jour conformément. C'est la boucle la plus rapide de prompt engineering qu'on puisse avoir.*

Récapitulatif

Ce qu'il faut retenir :

1. Ajouter un champ à un schéma Pydantic = **une ligne**, plus **un changement** du nom passé à `with_structured_output`.
2. La **description** d'un **Field** est une partie du prompt. Soigne-la (registre, public, forme).
3. Le coût marginal d'un champ ajouté est négligeable (~ 30 à 80 tokens supplémentaires).
4. Dans LangSmith, le tool schema reflète immédiatement les changements de description — boucle de prompt engineering ultra-rapide.

4 Router avec RunnableBranch et classifieur LLM

Exercice bonus 4. Aiguillage dynamique vers 3 sous-chaînes spécialisées

Crée un *router* qui choisit automatiquement entre 3 sous-chaînes selon que le concept Python est de catégorie **syntaxe** (def, lambda, comprehensions...), **paradigme** (POO, fonctionnel, polymorphisme...) ou **écosystème** (pip, venv, pytest...). Utilise une mini-chaîne LLM de classification en amont pour décider du routage.

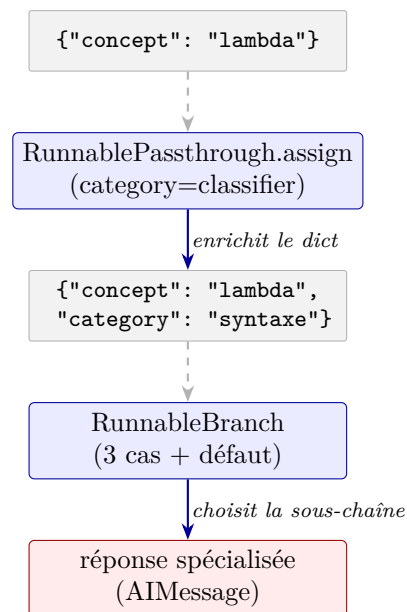
Contrainte : tout doit rester un seul Runnable LCEL pour préserver streaming, batching, et tracing LangSmith unifié.

Objectifs pédagogiques

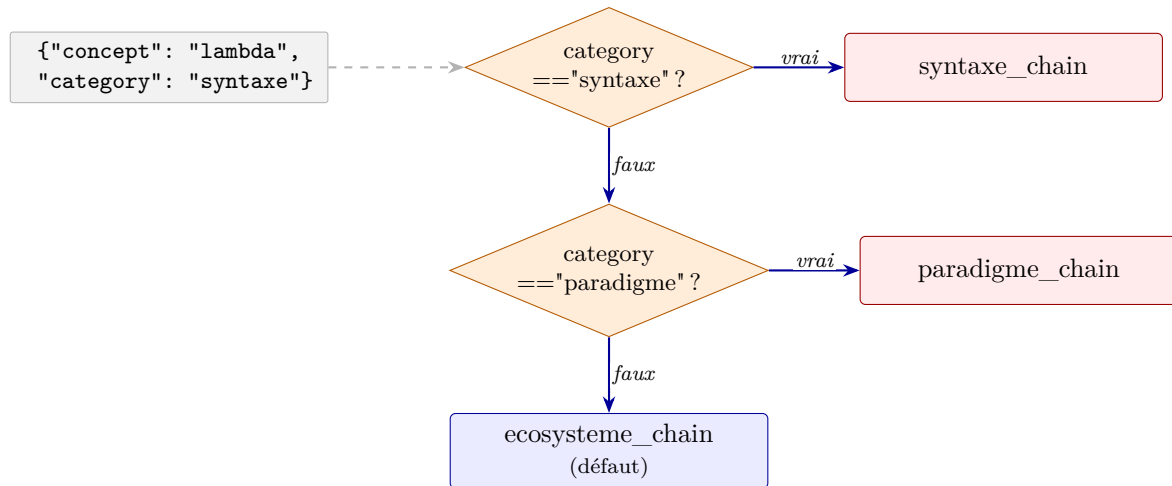
- Maîtriser le **pattern router** : classifieur LLM + sous-chaînes spécialisées + RunnableBranch.
- Utiliser `Literal[...]` dans un schéma Pydantic pour garantir un ensemble fermé de catégories.
- Combiner `RunnablePassthrough.assign` et `RunnableBranch` pour enrichir puis brancher.
- Repérer dans LangSmith les **trois appels LLM** qu'un router engendre par requête (et le surcoût associé).

4.1 Solution

4.1.1 Architecture



Zoom sur le RunnableBranch



4.1.2 Le code complet

```
1 from typing import Literal
2 from pydantic import BaseModel, Field
3 from langchain_core.prompts import ChatPromptTemplate
4 from langchain_core.runnables import (
5     RunnableBranch, RunnableLambda, RunnablePassthrough,
6 )
7 from config import get_model
8
9
10 class CategoryResult(BaseModel):
11     """Catégorie d'un concept Python pour router vers la bonne sous-chaîne."""
12     category: Literal["syntaxe", "paradigme", "ecosysteme"] = Field(
13         description=(
14             "Catégorise le concept :\n"
15             " - 'syntaxe' : éléments du langage (def, lambda, comprehensions, with, \
16             ↪ yield...)\n"
17             " - 'paradigme' : concepts de design (POO, fonctionnel, héritage, \
18             ↪ polymorphisme...)\n"
19             " - 'ecosysteme' : outils/bibliothèques (pip, venv, pytest, numpy, \
20             ↪ requests...)"
21         )
22     )
23
24 def build_classifier():
25     """Mini-chaîne qui catégorise un concept en 3 classes."""
26     classify_prompt = ChatPromptTemplate.from_messages([
27         ("system", "Tu classes des concepts Python en 3 catégories. "
28             "Ne réponds que par la catégorie."),
29         ("human", "Concept à catégoriser : {concept}"),
30     ])
31     return (
32         classify_prompt
33         | get_model().with_structured_output(CategoryResult)
34         | RunnableLambda(lambda r: r.category)
35     )
36
37 def build_specialized_chain(focus: str):
```

```

37     """Construit une sous-chaîne avec un prompt système orienté `focus`."""
38     prompt = ChatPromptTemplate.from_messages([
39         ("system",
40          "Tu es un assistant Python. Le concept à expliquer relève de la "
41          f"catégorie '{focus}'. Adapte ton explication en conséquence :\n" +
42          {
43              "syntaxe": "insiste sur la syntaxe exacte, montre un mini exemple et un
44                        ↪ anti-exemple.",
45              "paradigme": "insiste sur la philosophie, le 'pourquoi', les implications
46                        ↪ de design.",
47              "ecosysteme": "insiste sur l'usage pratique, l'installation, les commandes
48                        ↪ shell typiques.",
49          }[focus]),
50         ("human", "Explique : {concept}"),
51     ])
52     return prompt | get_model()
53
54 def build_router():
55     """Le routeur complet : classifie puis branche."""
56     classifieur = build_classifieur()
57     syntaxe_chain = build_specialized_chain("syntaxe")
58     paradigme_chain = build_specialized_chain("paradigme")
59     ecosysteme_chain = build_specialized_chain("ecosysteme")
60
61     branch = RunnableBranch(
62         (lambda x: x["category"] == "syntaxe", syntaxe_chain),
63         (lambda x: x["category"] == "paradigme", paradigme_chain),
64         ecosysteme_chain, # défaut : si rien ne matche
65     )
66
67     return RunnablePassthrough.assign(category=classifieur) | branch

```

4.1.3 Explications pas à pas

Étape 1 — Le vocabulaire de routage (CategoryResult)

Avant de router, il faut un **ensemble fermé** de catégories. Le `Literal["syntaxe", "paradigme", "ecosysteme"]` est crucial : il *garantit* que le LLM ne pourra pas renvoyer "truc" ou "je sais pas". La `description` du champ sert de mini-prompt qui explique au modèle *comment* classer.

Étape 2 — Le classifieur

Trois maillons en pipeline :

1. `classify_prompt : {"concept": "lambda"} → messages formatés.`
2. `model.with_structured_output(CategoryResult) : messages → objet CategoryResult(category="syntaxe")`
3. `RunnableLambda(lambda r: r.category) : objet → chaîne "syntaxe".`

Le maillon 3 n'est pas cosmétique : il rend la sortie utilisable comme **valeur de comparaison** dans les prédicats du `RunnableBranch`.

Étape 3 — Les sous-chaînes spécialisées

`build_specialized_chain(focus)` est une *fabrique* : on l'appelle 3 fois avec des `focus` différents et on obtient 3 `Runnable` distincts, chacun avec sa personnalité dans le `system prompt`.

*La spécialisation se fait à la **construction**, pas à l'appel. `build_specialized_chain("syntaxe")` renvoie un `Runnable` prêt à l'emploi qui ne sait répondre que dans le style « *syntaxe* ».*

Étape 4 — Le RunnableBranch

RunnableBranch est le if/elif/else de LCEL. Trois points clés :

- Les prédicats sont évalués **dans l'ordre**, le premier qui renvoie **True** gagne (court-circuit).
- Si *aucun* prédicat ne matche, la chaîne par défaut s'exécute — elle est **obligatoire**.
- La sous-chaîne sélectionnée reçoit **l'input complet** (le dict entier), pas juste le résultat du prédicat.

Étape 5 — L'enrichissement avant branchement

Le RunnableBranch a besoin que **category** soit **déjà dans l'input** pour que ses prédicats puissent le lire. C'est le rôle de RunnablePassthrough.assign :

*RunnablePassthrough.assign(category=classifier) appelle le classifieur et ajoute le résultat sous la clé **category**, sans toucher aux clés existantes. C'est l'équivalent d'un **SELECT *, expr AS col** en SQL.*

4.2 Vérification dans LangSmith

L'arbre d'exécution

Pour le concept "lambda", on observe **3 appels LLM** :

```
RunnableSequence (router)
  RunnableAssign<category>
    classifieur ← LLM #1 (structured_output=CategoryResult)
      → "syntaxe"
    RunnableBranch
      branch[0]: syntaxe_chain
    ChatAnthropic ← LLM #2 (réponse spécialisée)
```

Les quatre choses à regarder

1. **L'arbre lui-même** : tu vois *visuellement* le passage par le classifieur puis la branche choisie. C'est l'aiguillage dynamique en image.
2. **Premier ChatAnthropic** (classifieur) : onglet *Output* → l'objet **CategoryResult** et le tool-call que Claude a utilisé.
3. **Deuxième ChatAnthropic** (réponse spécialisée) : le system prompt commence par « Tu es un assistant Python...catégorie 'syntaxe' ». Précieux pour debugger.
4. **Onglet Costs** : tokens et coût agrégés sur les 2 (ou 3, si on garde le **debug_chain**) appels. Permet de chiffrer le surcoût du pattern router.

*Compare deux runs dans l'UI : "lambda" déclenche **syntaxe_chain**, "pytest" déclenche **ecosysteme_chain**. On voit littéralement **la branche choisie** changer selon le concept.*

Récapitulatif

Ce qu'il faut retenir :

1. **Pattern router** = classifieur LLM + sous-chaînes spécialisées + **RunnableBranch**.
2. **with_structured_output + Literal** = garantie que la sortie de classification est dans un ensemble fermé, donc routable sans validation supplémentaire.

3. `RunnablePassthrough.assign(...)` enrichit le dict de contexte au fil de la pipeline, sans perdre les clés précédentes.
4. Le tout reste un **seul Runnable** : `router.invoke`, `router.batch`, `router.astream` fonctionnent — et LangSmith trace l'arbre complet en un seul run.
5. Surcoût : **2 appels LLM par requête** (classification + réponse), au lieu d'un seul avec un prompt méga-universel. Acceptable pour gagner en qualité, mesurable dans LangSmith.

5 Async avec `asyncio.gather` vs `RunnableParallel`

Exercice bonus 5. La voie async pure et sa comparaison avec `RunnableParallel`

Réécrit la démo multi-niveaux en **async** avec `await chain.ainvoke(...)` et `asyncio.gather(...)`. Compare les durées avec la version `RunnableParallel` (synchrone et asynchrone).

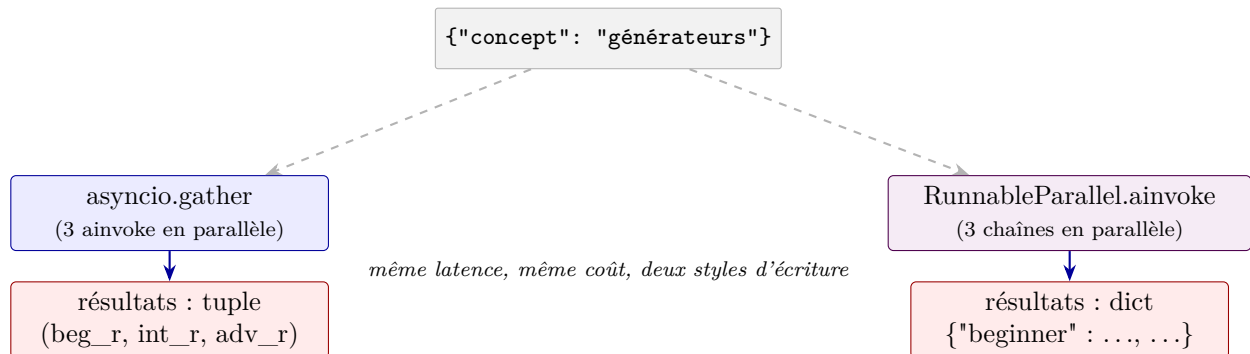
Contrainte : utiliser `asyncio.run()` comme point d'entrée pour la fonction **async**, et `await` sur `.ainvoke()` (la variante asynchrone de `.invoke()`).

Objectifs pédagogiques

- Connaître les méthodes `.ainvoke()`, `.abatch()`, `.astream()` — les variantes async de tous les Runnable.
- Comprendre que `RunnableParallel` utilise **asyncio sous le capot** : la version async et la version sync donnent des temps quasi identiques.
- Choisir entre `asyncio.gather` (contrôle explicite) et `RunnableParallel` (composition implicite).

5.1 Solution

5.1.1 Architecture



Les deux approches s'appuient sur le **même runtime asyncio** : event loop unique, 3 coroutines en concurrence, attente non bloquante. La différence est purement stylistique.

5.1.2 Le code complet

```

1 import asyncio
2 import time
3 from langchain_core.runnables import RunnableParallel
4 from phase1_explain import explain_chain_for_level
5
6
7 async def exercise_5() -> None:
8     concept = "les générateurs Python"
  
```

```

9     beginner      = explain_chain_for_level("débutant")
10    intermediate   = explain_chain_for_level("intermédiaire")
11    advanced       = explain_chain_for_level("avancé")
12
13    # --- Version asyncio.gather pure ---
14    t0 = time.perf_counter()
15    beg_r, int_r, adv_r = await asyncio.gather(
16        beginner.ainvoke({"concept": concept}),
17        intermediate.ainvoke({"concept": concept}),
18        advanced.ainvoke({"concept": concept}),
19    )
20    gather_time = time.perf_counter() - t0
21    print(f"asyncio.gather      : {gather_time:.2f}s")
22
23    # --- Version RunnableParallel avec .ainvoke (async aussi) ---
24    parallel = RunnableParallel(
25        beginner=beginner, intermediate=intermediate, advanced=advanced,
26    )
27    t0 = time.perf_counter()
28    results = await parallel.ainvoke({"concept": concept})
29    parallel_time = time.perf_counter() - t0
30    print(f"RunnableParallel    : {parallel_time:.2f}s")
31
32    # --- Version synchrone pour référence ---
33    t0 = time.perf_counter()
34    parallel.invoke({"concept": concept})
35    sync_time = time.perf_counter() - t0
36    print(f"Sync (RunnablePar): {sync_time:.2f}s")
37
38
39    # Point d'entrée : asyncio.run lance la coroutine
40    if __name__ == "__main__":
41        asyncio.run(exercise_5())

```

5.1.3 Explications pas à pas

Étape 1 — Le préfixe a : la version async de tout

LCEL définit une variante async pour chaque méthode synchrone d'un Runnable :

Synchrone	Asynchrone
.invoke(input)	.ainvoke(input)
.batch(inputs)	.abatch(inputs)
.stream(input)	.astream(input)

Les versions async retournent des coroutines, à attendre avec `await` dans un contexte `async def`.

Étape 2 — `asyncio.gather` pur

`asyncio.gather(*coros)` prend N coroutines et les exécute en **concurrency** sur l'évent loop. `await` sur le résultat attend que toutes terminent et renvoie un **tuple** des résultats dans l'ordre des arguments :

```

1 beg_r, int_r, adv_r = await asyncio.gather(
2     beginner.ainvoke({"concept": concept}),      # → beg_r
3     intermediate.ainvoke({"concept": concept}),  # → int_r
4     advanced.ainvoke({"concept": concept}),      # → adv_r

```

C'est l'équivalent *asynchrone* d'un `ThreadPoolExecutor` : le main thread ne fait rien pendant que les 3 appels HTTP sont en vol.

Étape 3 — `RunnableParallel` utilise `asyncio` sous le capot

Quand on appelle `parallel.invoke(...)` (synchrone), LCEL crée en interne un event loop, lance les branches comme des coroutines, et attend le tout. Quand on appelle `parallel.ainvoke(...)`, on réutilise l'event loop existant — plus efficace, surtout dans une app web déjà en async.

Conséquence pratique : `parallel.invoke()` et `await parallel.ainvoke()` donnent des temps identiques à la milliseconde près. Le choix est dicté par le contexte d'appel (synchrone ou asynchrone), pas par la performance.

Étape 4 — Quand utiliser `gather` vs `RunnableParallel`

<code>asyncio.gather</code>	<code>RunnableParallel</code>
Bibliothèque standard Python.	Primitive LCEL.
Contrôle explicite du flot de coroutines.	Composition déclarative dans une chaîne.
Bien pour du code applicatif : routes FastAPI, boucle de chat, jobs.	Bien pour la <i>composition</i> dans un <code>Runnable</code> (qui sera lui-même streamé, batché, tracé).
Pas de tracing LangSmith par défaut (faut envelopper manuellement).	Tracing LangSmith automatique.

*Règle pratique : `RunnableParallel` dans les **chaînes** (parce que c'est composable et tracé); `asyncio.gather` dans le **code applicatif** (parce que c'est explicite et standard Python).*

Étape 5 — Le point d'entrée `asyncio.run`

Une fonction `async def` ne s'exécute pas avec un appel direct : elle renvoie une coroutine non encore lancée. `asyncio.run(coro)` crée un event loop, exécute la coroutine, et ferme proprement.

```

1 async def exercise_5() -> None:
2     ...                               # contient des await
3
4 # Dans le main, on emballe avec asyncio.run
5 if __name__ == "__main__":
6     asyncio.run(exercise_5())

```

5.2 Vérification dans LangSmith

Les deux versions (`gather` et `RunnableParallel.ainvoke`) produisent des arbres très similaires dans LangSmith. La différence :

- `RunnableParallel.ainvoke` : un *seul* run racine `RunnableParallel` avec 3 enfants (un par branche).
- `asyncio.gather` avec 3 `.ainvoke()` séparés : **3 runs racines distincts**, chacun étant la trace d'un `RunnableSequence`. Pas d'agrégation automatique.

Pour avoir un seul run agrégé en `gather`, il faut envelopper manuellement avec `trace` de `langsmith` :

```
1 from langsmith import trace
2
3 with trace("multi-level parallel"):
4     beg_r, int_r, adv_r = await asyncio.gather(...)
```

*La traçabilité **automatique** est un argument fort pour `RunnableParallel`. Dès qu'on sort de LCEL, le tracing devient manuel.*

Récapitulatif

Ce qu'il faut retenir :

1. Tout `Runnable` expose une variante *async* : `.ainvoke`, `.abatch`, `.astream`. Le préfixe *a* = *async*.
2. `asyncio.gather(*coros)` lance *N* coroutines en concurrence et renvoie un tuple des résultats dans l'ordre.
3. `RunnableParallel` utilise *asyncio* sous le capot — `.invoke()` et *await* `.ainvoke()` ont les mêmes performances.
4. Règle pratique : `RunnableParallel` dans les chaînes (pour la composition et le tracing), `asyncio.gather` dans le code applicatif (pour le contrôle explicite).
5. `asyncio.run(coro)` est le point d'entrée pour appeler une coroutine depuis un contexte synchrone (*script*, `if __name__ == "__main__"`).